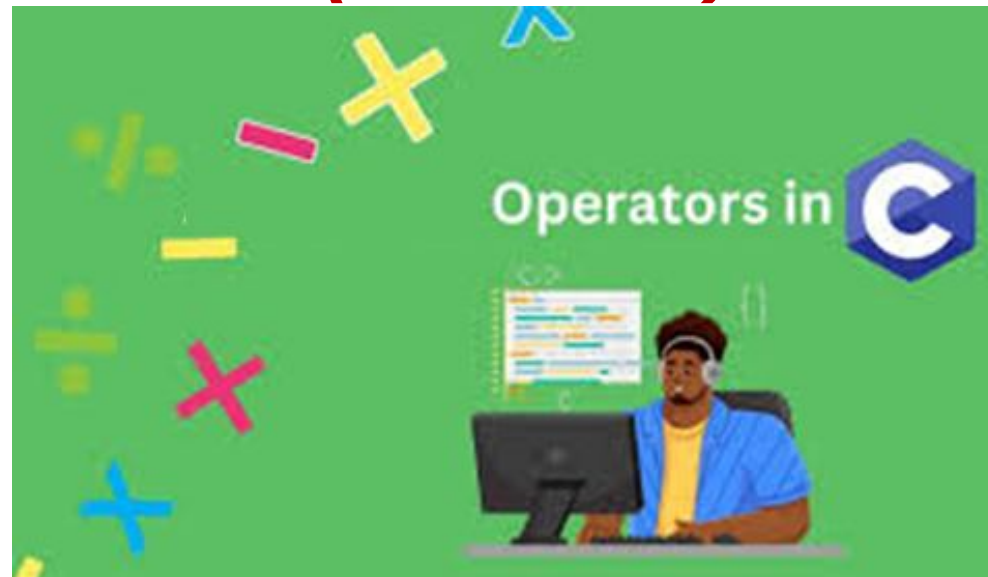




Structured Programming Concepts

(26CSE0105)



Operators in C

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

- ***Operators in C***
 - ***Arithmetic operators***
 - ***Relational operators***
 - ***Logical operators***
 - ***Assignment operator***
 - ***increment/decrement operators***
 - ***bitwise operator***

Operators in C



- An **operator** is a **symbol** that **operates** on a **variable** or **value**.
- The **values** and **variables** used with **operators** are called **operands**.
- **Operators** can be classified into **three types** on the basis of the **number of operands**.

	Operators	Type
Unary Operator →	++, -	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Rational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	Size of, comma (,), conditional(?:) dot (.), arrow (->), cast (type), addressof (&), dereference (*)	Other Operator

Types of operators in C



Arithmetic Operators in C

- Classified into **two types** based on the **number of operands**:
 - ❖ **Binary Arithmetic Operators**
 - ❖ **Unary Arithmetic Operators**

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Arithmetic Operators in C

- While **executing an arithmetic statement** that has **multiple operators**, there might be **some concerns** about their evaluation.
- For these concerns, we need to understand;

“The **priority** or **precedence** in which the **operations** in an **arithmetic statement** are performed”.

Priority	Operators	Description
1 st	* / %	Multiplication, Division, Modular division
2 nd	+ -	Addition, Subtraction
3 rd	=	Assignment

Arithmetic Operators in C

- When an expression contains **two operators** of **equal priority** then **tie between them** is settled using the **associativity** of the **operators**.
- All **operators in C** have;
 - either **Left to Right associativity**.
 - or **Right to Left associativity**.

Description	Operator	Associativity
Multiplication	*	Left to right
Division	/	Left to right
Modulus	%	Left to right
Addition	+	Left to right
Subtraction	-	Left to right

Scanning & Evaluation in Compiler

- **Scanning** and **Evaluation** are **two separate stages**.

Scanning:

- **Compiler** scans the statement from **left to right** and identifies **tokens**.

Evaluation:

- **Compiler** groups and evaluates operators **according to precedence** and **associativity**; therefore, **evaluation** is not simply **left-to-right**.

- **Compiler reads/scans** the expression from **left to right** to identify tokens.
- But it **does not simply calculate** everything from **left to right**. i.e., (**operators** of **same precedence** are **grouped** from **left to right**).

Scanning Version (example)

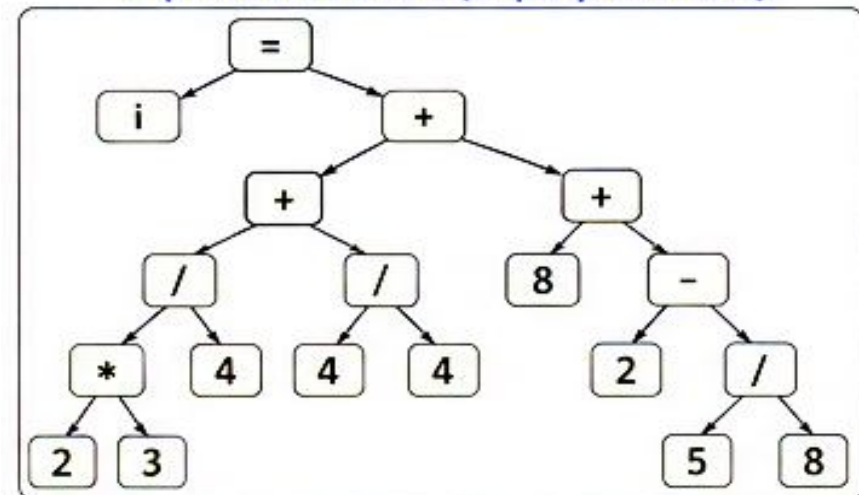
Example: *Evaluate* the following expression, assuming that *i* is an *integer variable*:

$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

The compiler goes through the following phases:

Phase 1 Lexical Analysis (Scanning)	Reads the source code from left to right and breaks it into tokens (lexemes).
Phase 2 Syntax Analysis (Parsing)	Checks the grammatical structure and builds the structure of the expression as per operator precedence and associativity.
Phase 3 Semantic Analysis	Checks meaning of expression, data types, type compatibility, etc.
Phase 4 Intermediate Code Generation	Converts the expression into an intermediate form (e.g., postfix).
Phase 5 Code Generation	Generates target (machine) code and executes the program.

Expression Structure (as per precedence)



Postfix (Intermediate Code)

i 2 3 * 4 / 4 4 / + 8 2 - 5 8 / + + =

Compilation Result

No errors.
Program compiled successfully. ✓

Execution Result (Value Stored)

i = 8

Evaluation Version (example)

Example: *Evaluate* the following expression, assuming that *i* is an *integer variable*:

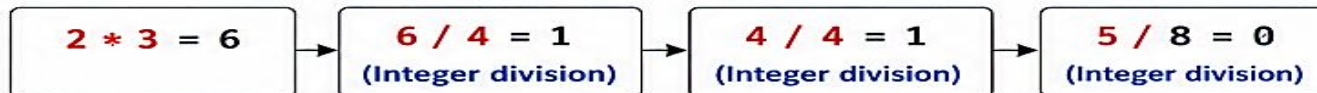
$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

Step 1: Apply Operator Precedence

- *** and */* have higher precedence than *+* and *-*.
- Operators with the same precedence are evaluated from left to right,

$$i = (2 * 3 / 4) + (4 / 4) + 8 - 2 + (5 / 8)$$

Step 2: Evaluate * and */* from left to right**



Step 3: Substitute the results

$$i = 1 + 1 + 8 - 2 + 0$$

Step 4: Evaluate *+* and *-* from left to right



Step	Expression	Result
1	$2 * 3$	6
2	$6 / 4$	1
3	$4 / 4$	1
4	$5 / 8$	0
5	$1 + 1$	2
6	$2 + 8$	10
7	$10 - 2$	8
8	$8 + 0$	8
Final	$i = 8$	$i = 8$

Final Result

$$i = 8$$

Important C Rule

When both operands of */* are integers, C performs **integer division**.

Thus,

$$6 / 4 \text{ gives } 1$$

$$5 / 8 \text{ gives } 0$$

Relational Operators in C

- Used to **compare two values** (or **variables**). i.e. **binary operators**.
- Help us to **find answers** and **make decisions**.
- The **return value** of a **comparison** is either **1** (**True**) or **0** (**False**). i.e. **Boolean values**.

Operator	Name	Example	Description
==	Equal to	x == y	Returns 1 if the values are equal
!=	Not equal	x != y	Returns 1 if the values are not equal
>	Greater than	x > y	Returns 1 if the first value is greater than the second value
<	Less than	x < y	Returns 1 if the first value is less than the second value
>=	Greater than or equal to	x >= y	Returns 1 if the first value is greater than, or equal to, the second value
<=	Less than or equal to	x <= y	Returns 1 if the first value is less than, or equal to, the second value

Relational Operators in C

```
#include <stdio.h>
int main()
{
    int a = 25, b = 5;
    printf("a < b : %d\n", a < b);
    printf("a > b : %d\n", a > b);
    printf("a <= b: %d\n", a <= b);
    printf("a >= b: %d\n", a >= b);
    printf("a == b: %d\n", a == b);
    printf("a != b: %d\n", a != b);
    return 0;
}
```

```
a < b : 0
a > b : 1
a <= b: 0
a >= b: 1
a == b: 0
a != b: 1
```

Logical Operators in C

- Used to **determine the logic** between **variables** or **values**, by **combining multiple conditions**.
- Result of the operation of a **logical operator** is either **True** or **False**.

Operator	Name	Example	Description
&&	AND	<code>x < 5 && x < 10</code>	Returns 1 if both statements are true
	OR	<code>x < 5 x < 4</code>	Returns 1 if one of the statements is true
!	NOT	<code>!(x < 5 && x < 10)</code>	Reverse the result, returns 0 if the result is 1

Logical Operators in C

```
#include<stdio.h>
int main()
{
    int a = 25, b = 5;
    printf("a && b : %d\n", a && b);
    printf("a || b : %d\n", a || b);
    printf("!a    : %d\n", !a);
    return 0;
}
```

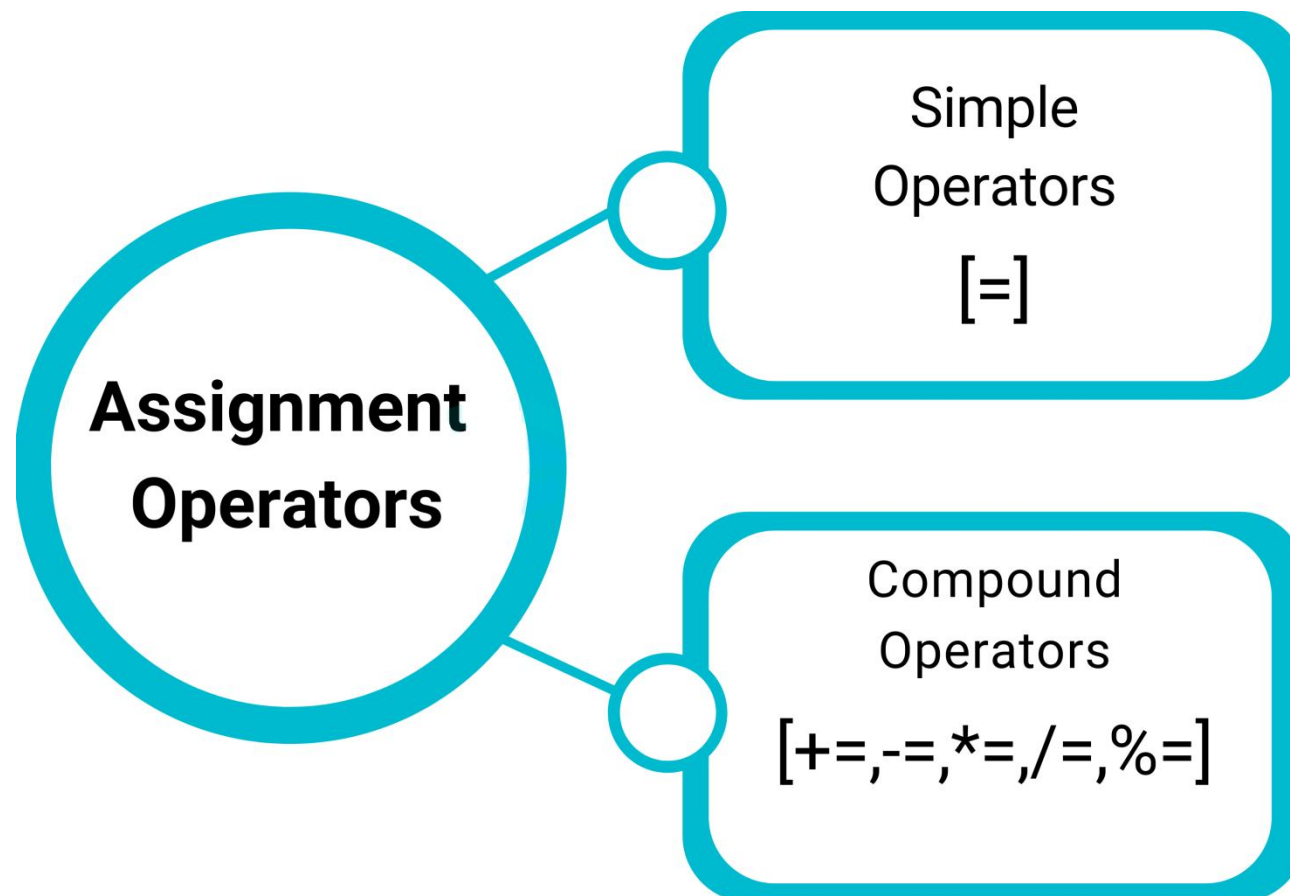
```
a && b : 1
a || b : 1
!a      : 0
```

```
...Program finished with exit code 0
Press ENTER to exit console. □
```

Assignment Operator in C



- Used to **assign value to a variable**.
- The **assignment operators** can be **combined** with some other operators to provide **multiple operations** using **single operator**.



Assignment Operator in C

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Assignment Operators in C

```
#include<stdio.h>
int main()
{
    int a = 25, b = 5;
    printf("a = b : %d\n", a = b);
    printf("a += b: %d\n", a += b);
    printf("a -= b: %d\n", a -= b);
    printf("a *= b: %d\n", a *= b);
    printf("a /= b: %d\n", a /= b);
    printf("a %= b: %d\n", a %= b);
    printf("a &= b: %d\n", a &= b);
    return 0;
}
```

```
a = b : 5
a += b: 10
a -= b: 5
a *= b: 25
a /= b: 5
a %= b: 0
a &= b: 0
```

```
...Program finished with exit code 0
Press ENTER to exit console. 
```

Bitwise Operator in C

- Used to perform **bit-level operations** on the **operands**.
- The **operators** are first converted to **bit-level** and then the **calculation is performed** on the **operands**.

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR (XOR)
<<	Bitwise Left Shift
>>	Bitwise Right Shift
~	One's Complement

Bitwise Operator in C

```
#include<stdio.h>
int main()
{
    int a = 12, b = 5;
    printf("a & b: %d\n", a & b);
    printf("a | b: %d\n", a | b);
    printf("a >> 2: %d\n", a >> 2);
    printf("a << 2: %d\n", a << 2);
}
```

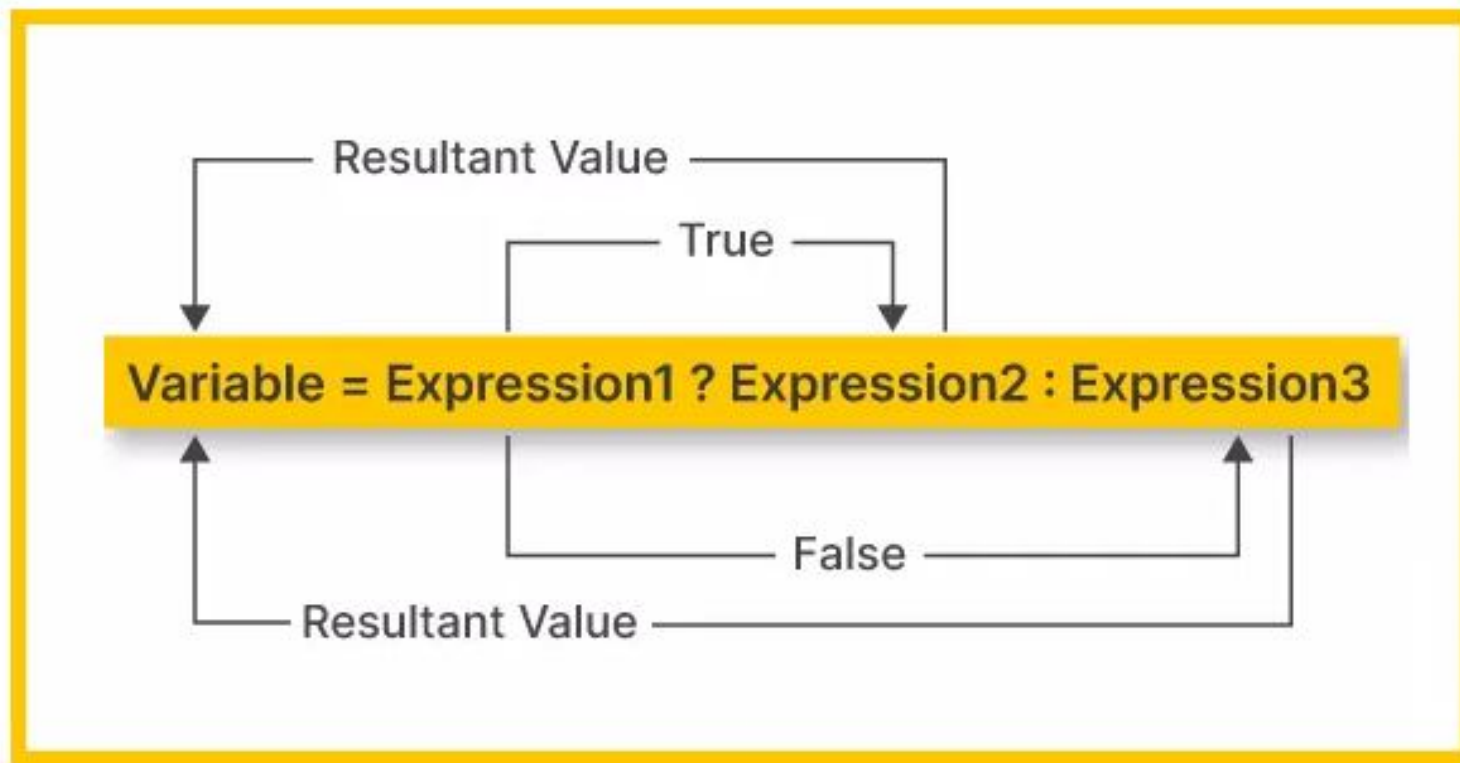
```
a & b: 4
a | b: 13
a >> 2: 3
a << 2: 48
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Ternary Operator in C



- It is a **conditional operator** that we can use in place of **if-else statements**.



Addressof Operator in C

- **Addressof** (&) Operator returns the **address of a variable**.

```
#include<stdio.h>
int main()
{
    int num = 10;
    printf("sizeof(num) = %d bytes\n", sizeof(num));
    printf("&num = %d\n", &num);
    return 0;
}
```

```
sizeof(num) = 4 bytes
&num = 184228556

...Program finished with exit code 0
Press ENTER to exit console. □
```



Thank You!

